

src/main.ts, line by line

A beginner-friendly guide to how this NestJS backend starts

What this file does

This is the backend entry point. It creates the NestJS application, enables global validation for incoming request data, and starts listening for HTTP requests.

The complete file

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module.js';
import { ValidationPipe } from '@nestjs/common';

async function bootstrap() {
  const app = await NestFactory.create(AppModule, {});
  app.useGlobalPipes(new ValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
    transform: true,
  }));
  await app.listen(process.env.PORT ?? 3000);
}
await bootstrap();
```

Big picture: import tools -> build the application -> configure input checks -> open the HTTP listener.

1. Imports and bootstrap

Line 1

```
import { NestFactory } from '@nestjs/core';
```

import makes code from another module available here. NestFactory is NestJS's application factory: it creates the server application.

Line 2

```
import { AppModule } from './app.module.js';
```

AppModule is the root module that tells Nest which application modules, controllers, and providers to load. The .js extension is intentional: this project uses Node ESM/NodeNext module rules, so compiled JavaScript imports use .js filenames.

Line 3

```
import { ValidationPipe } from '@nestjs/common';
```

ValidationPipe is Nest's built-in request-data processing tool. Here it will validate applicable input before the route handler runs.

Line 5

```
async function bootstrap() {
```

This declares the startup function. async allows await inside it. Creating and starting a network server involve asynchronous operations, so the function waits for those operations in sequence.

Line 6

```
const app = await NestFactory.create(AppModule, {});
```

- NestFactory.create builds a Nest application using AppModule as its root.
 - await waits for application setup to finish before the next line runs.
 - const app stores the resulting application object, used below to configure and start the server.
 - The second argument is an empty options object, so Nest uses default options.
- In plain English: ?Build my backend using the application structure described by AppModule.?

2. Global request validation

Line 7

```
app.useGlobalPipes(new ValidationPipe({
```

A pipe processes data before a controller method receives it. `useGlobalPipes` registers this pipe for the whole application, rather than just one route.

For example, a POST `/staff` request body is checked before `StaffController.create()` is called.

Line 8: `whitelist`

```
whitelist: true,
```

Only properties that have validation decorators on the route DTO are retained. For this application, the staff creation DTO defines the accepted request fields.

Line 9: `forbidNonWhitelisted`

```
forbidNonWhitelisted: true,
```

Instead of silently removing an unrecognized field, reject the request. This gives the caller an explicit validation error if they send a field the DTO does not allow.

Example: if a create-staff request contains `status` but `CreateStaffDto` has no `status` property with validation rules, this setting makes the request invalid.

Line 10: `transform`

```
transform: true,
```

Asks Nest to transform plain incoming data into the DTO class instance and apply eligible type conversions. This works together with the DTO and its decorators.

Lines 11-12: closing calls

```
});
```

The first closing parenthesis finishes `new ValidationPipe({...})`; the second finishes

`app.useGlobalPipes(...)`. Together they register one global pipe with the three options.

Important: these options do not define every field rule. Rules such as `?firstName must be a string?` are written on `CreateStaffDto`. Validation also does not authenticate users or decide their permissions.

3. Start listening for requests

Line 13

```
await app.listen(process.env.PORT ?? 3000);
```

- app.listen starts the HTTP server so it can receive requests.
- process.env.PORT reads the PORT environment variable, if configured.
- ?? 3000 uses 3000 only when PORT is null or undefined.
- await waits for the listener to start before bootstrap completes.

Locally, this usually means the server is available at <http://localhost:3000>, unless a different PORT is configured.

Line 14

```
}
```

This closes the bootstrap function.

Line 15

```
await bootstrap();
```

Defining a function does not run it. This line calls bootstrap. The await at the top level is supported because this project uses ES modules.

What happens when a request arrives?

1. The HTTP server receives the request.
2. Nest matches the URL and method to a controller route.
3. Applicable input passes through the global ValidationPipe.
4. If valid, Nest calls the controller method.
5. The controller can call a service; the service can use Prisma to access the database.
6. The result is returned to the client as an HTTP response.

One-sentence summary

src/main.ts builds the Nest application from AppModule, rejects invalid request data using the configured global pipe, and starts the server on PORT or 3000.